AUTOMATED PLANNING

Automated Planning is an important problem solving activity which consists in synthesizing a ***sequence of actions*** performed by an agent that leads from an initial state of the world to a given target state (**goal**)

AUTOMATED PLANNING

Given:

- an initial state
- a set of actions you can perform
- a state to achieve (**goal**)

Find:

a plan: a partially or totally ordered set of actions needed to achieve the goal from the initial state

Planning is:

- one application per se
- A common activity in many applications such as
  - diagnosis: test plans and actions to repair (reconfigure) a system
  - scheduling
  - robotics

AUTOMATED PLANNING: basics

An **automated planner** is an *intelligent agent* that operates in a certain domain described by:

1. ***a representation of the initial state***
2. ***a representation of a goal***
3. ***a formal description of the executable actions (also called operators)***

It dynamically defines the plan of actions needed to reach the goal from the initial state.

ACTION REPRESENTATION

- A planner relies on a formal description of the executable actions. It is called ***Domain Theory***.
- Each action is identified by a name and declaratively modeled through ***preconditions*** is ***postconditions***.
- Preconditions are the conditions which must hold to ensure that the action can be executed;
- Postconditions represent the effects of the action on the world.
- Often the Domain Theory consists of operators containing variables that define **classes of actions**. A different instantiation of the variables corresponds to a different action.

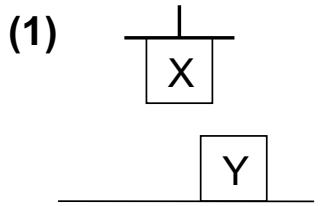
EXAMPLE: THE BLOCK WORLD

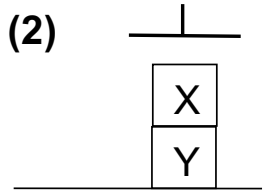
Actions:

STACK (X, Y)

IF: holding (X) and clear (Y)

THEN: handempty and clear (X) and on (X, Y);

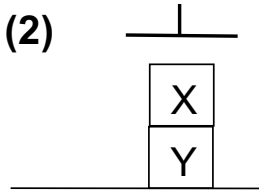


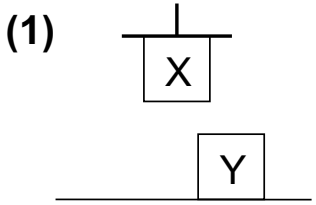


UNSTACK (X, Y)

IF: handempty and clear (X) and on (X, Y)

THEN: holding (X) and clear (Y);

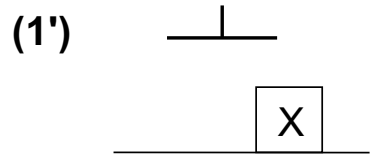




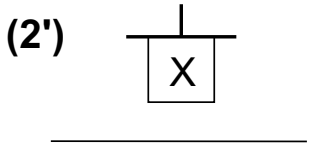
EXAMPLE: THE BLOCK WORLD

PICKUP (X)

IF: ontable (X) and clear (X) and handempty

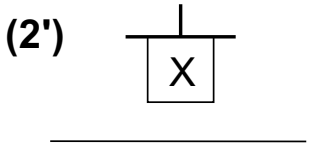


THEN: holding (X);

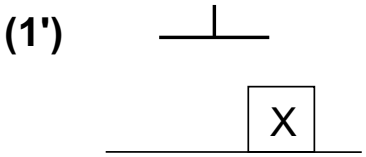


Putdown (X)

IF: holding (X)



THEN: ontable (X) and clear (X) and handempty.



DEDUCTIVE PLANNING

Deductive planning uses logics for representing states, goals and actions and generates a plan as a theorem proof.

Green and Kowalsky formulations

SITUATION CALCULUS

First order logic to describe states and clauses to describe actions

- **Situation**: world snapshot describing properties (*fluents*) that hold in a given state s
  - Example: block world

    on(b,a,s)

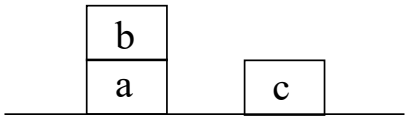
    ontable(c,s)

- **Actions**: define which fluents are true as a consequence of an action

  on(X,Y,S) and clear(X,S) $\rightarrow$

  (ontable(X,do(putOnTable(X),S))) *and*

  (clear(Y,do(putOnTable(X),S)))



DEDUCTIVE PLANNING

Green uses *situation calculus* to build a planning based on logic resolution.

He finds a proof of a formula containing a state variable. At the end of the proof the state variable will be instantiated to the plan to reach the objective

SITUATION CALCULUS

- ***Plan construction*** *: deduction, goal proof*
  - *Example*

    :- ontable(b,S). Means: does a state S exist in which ontable(b) is true?

    YES with S=putOntable(b,s)

- ***Advantages****: high expressivity, can describe complex problems*

- ***Limitations****: frame problem*

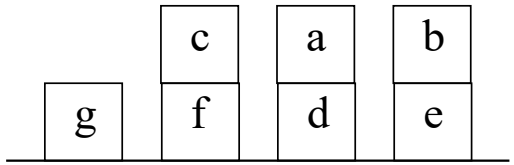
FRAME PROBLEM

- Knowledge representation problem
- We have to explicitly list all fluents that change and that DO NOT change after a state transition.
- If we have a complex domain the number of these axioms grows enormously.

EXAMPLE

- These axioms describe propositions that are true in the initial state s0

A.1 *on(a,d,s0).*
A.2 *on(b,e,s0).*
A.3 *on(c,f,s0).*
A.4 *clear(a,s0).*
A.5 *clear(b,s0).*
A.6 *clear(c,s0).*
A.7 *clear(g,s0).*
A.8 *diff(a,b)*
A.9 *diff(a,c)*
A.10 *diff(a,d)…*



EXAMPLE

- Actions are clauses

Action *move(X,Y,Z)*

clear(X,S) and clear(Z,S) and on(X,Y,S) and diff(X,Z)
→ *clear(Y,do(move(X,Y,Z),S)), on(X,Z,do(move(X,Y,Z),S))*.

moves a block X from Y to Z, starting from a state S and ending in a state *do(move(X,Y,Z),S)*) that translates in the following axioms (*effect axioms)*

A.11 *~clear(X,S)* or *~clear(Z,S)* or *~on(X,Y,S)* or *~diff(X,Z) or clear(Y,do(move(X,Y,Z),S))*.

A.12 *~clear(X,S)* or *~clear(Z,S)* or *~on(X,Y,S)* or *~diff(X,Z) or on(X,Z,do(move(X,Y,Z),S))*.

EXAMPLE

Given a goal we find a proof through the resolution process:

GOAL:- on(a,b,S1)

~on(a,b,S1)

(A.12) {X/a,Z/b,S1/do(move(a,Y,b),S)}

~clear(a,S) or ~clear(b,S) or ~on(a,Y,S) or ~diff(a,b)

(A.4)	*(A.5)*	*(A.1)*	*(A.8)*
{S/s0},	*{S/s0},*	*{S/s0, Y/d}*	*true*

~on(a,b,S1) leads to a contraddiction to we have a proof for on(a,b,S1) with the substitution **S1/do(move(a,d,b),s0).** This is our plan!!

Now suppose we want to solve a more complex problem

Goal: on(a,b,S), on(b,g,S).

Solution*:* ***S/do(move(a,d,b),do(move(b,e,g),s0)).***

To solve this problem we need a complete description of the state after each move.

EXAMPLE

To describe an action, beside effect axioms we have to specify all fluents that would not change by the execution of an action (*frame axioms)*. In our example we would need:

on(U,V,S) and diff(U,X) → *on(U,V,do(move(X,Y,Z),S))*
clear(U,S) and diff(U,Z) → *clear(U,do(move(X,Y,Z),S))*

We need a frame axiom for each condition that is not changed by each action.



If the problem is complex, too many axioms

KOWALSKY FORMULATION

- We use the predicate *holds(rel, s/a)* to describe all the relations rel that are true in a given state s or made true by the eecution of an action.
- Predicate *poss(s)* that indicates if a state *s is possible* (namely reachable).
- Predicate *pact(a,s)* to indicate that it is possible to execute an action A in a state s, namely the preconditions of a are true in s.

If a stateA is possible S and the preconditions of an action A are satisfied, then it is possible the state produced by S after the execution of A:

poss(S) and pact(A,S) → poss(do(A,S))

KOWALSKY FORMULATION

We need **one frame assertion per action** (advantage w.r.t Green)

In the previous example we would have

holds(V,S) $\wedge$ *diff(V,clear(Z))* $\wedge$ *diff(V,on(X,Y))* $\rightarrow$
holds(V,do(move(X,Y,Z),S)))

that states that every term different from clear(Z) and on (X,Y) (**delete list**) are true after the eecution of the action move.

EXAMPLE

Goal

:- *poss(S), holds(on(a,b),S), holds(on(b,g),S)*.

Use PROLOG to solve it

Initial State

poss(s0).

holds(on(a,d),s0).

holds(on(b,e),s0).

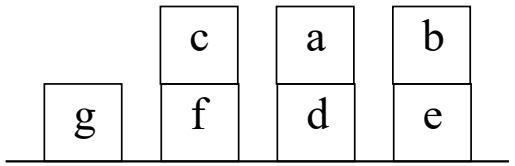
holds(on(c,f),s0).

holds(clear(a),s0).

holds(clear(b),s0).

holds(clear(c),s0).

holds(clear(g),s0).



EXAMPLE

Effects of move(X,Y,Z):

holds(clear(Y),do(move(X,Y,Z),S)).

holds(on(X,Z),do(move(X,Y,Z),S)).

Preconditions of move(X,Y,Z):

pact(move(X,Y,Z),S):-

holds(clear(X),S), holds(clear(Z),S), holds(on(X,Y),S), X\=Z.

Frame conditions:

holds(V,do(move(X,Y,Z),S)):- holds(V,S), V\=clear(Z), V\=on(X,Y).

Clause for state reachability

poss(do(A,S)):- poss(S), pact(A,S).

Goal :- *poss(S), holds(on(b,g),S), holds(on(a,b),S)*.

Yes per *S* = ***do(move(a, d, b), do(move(b, e, g), s0))***